

AI DECEPTION ANALYZER (CLI-BASED)

Fundamentals of AI and ML – Course Project

Submitted By

Name: Ritesh Kumar Verma

Reg No: 25BAI11426

Course: Fundamentals of AI and ML (CSA2001)

GitHub Repository:

<https://github.com/codeliferitesh/ai-deception-analyzer>

1. TITLE

AI Deception Analyzer: A CLI-Based Machine Learning System for Detecting Deceptive Text and Emotional Tone

2. INTRODUCTION

Artificial Intelligence (AI) and Machine Learning (ML) are becoming a major part of how systems understand human language today. From chatbots to recommendation systems, AI is everywhere.

One interesting problem is detecting whether a statement is **truthful or deceptive**. Humans often struggle with this, especially when the language is emotional or indirect.

This project, **AI Deception Analyzer**, is a simple yet effective **Command Line Interface (CLI)** application that analyzes user input text and predicts:

- Whether the statement is truthful or deceptive 🤔
- The emotional tone behind the text 😊 😡 😐

The system uses **Natural Language Processing (NLP)** techniques along with a machine learning model to perform this analysis. The main goal was to build something that is:

- Lightweight ⚡
- Easy to run in terminal 💻
- Practical and real-world focused 🌍

🚀 3. MOTIVATION OF THE PROJECT

In everyday communication, it is not easy to detect deception. People may:

- Over-explain things
- Use emotional language
- Avoid direct answers

This made me curious — *can a machine detect such patterns?*

The motivation behind this project includes:

- Exploring real-world applications of AI 🤖
- Understanding how NLP works in practice
- Building a complete CLI-based system (as required)
- Applying classroom concepts to a practical problem

! 4. PROBLEM STATEMENT

Detecting deception in text is a challenging task because:

- There are no clear or fixed rules
- Language is subjective and varies from person to person
- Emotional tone can hide the real intent

Problem:

Design a system that can analyze text input and predict whether it is **truthful or deceptive**, along with identifying the **emotion behind it**.

5. OBJECTIVE OF THE PROJECT

The main objectives of this project are:

- Build a complete CLI-based AI application 
 - Apply NLP techniques to process text 
 - Train a machine learning model for classification 
 - Detect emotional tone using simple logic   
 - Store analysis history for later use 
 - Ensure smooth and user-friendly CLI interaction
-

6. EXISTING METHODS

Some existing approaches for deception detection include:

1. Human Judgment

- Based on intuition and experience

2. Psychological Analysis

- Uses behavioral and linguistic cues

3. Advanced AI Models (e.g., BERT)

- Deep learning-based approaches
-



7. PROS AND CONS OF EXISTING METHODS



Pros:

- Advanced models provide high accuracy
- Psychological methods consider context



Cons:

- Human judgment is often unreliable
 - Deep learning models are complex and heavy
 - Not suitable for simple CLI applications
-



8. HARDWARE AND SOFTWARE REQUIREMENTS



Hardware:

- Basic laptop or computer
- Minimum 4GB RAM



Software:

- Python 3.8+
- VS Code / Terminal



Libraries:

- scikit-learn
 - pandas
 - numpy
 - joblib
-

9. METHODOLOGY AND GOAL

Methodology:

1. Collect sample dataset
2. Preprocess text data
3. Convert text using TF-IDF
4. Train model using Logistic Regression
5. Detect emotion using keyword-based logic
6. Build CLI for user interaction

Goal:

To create a **simple, fast, and practical AI system** that works efficiently in a terminal environment.

10. FUNCTIONAL MODULES DESIGN AND ANALYSIS

- **Input Module:** Takes user input
- **Processing Module:** Converts text to features
- **Prediction Module:** Predicts deception
- **Emotion Module:** Detects emotion

- **Storage Module:** Saves results
 - **CLI Module:** Handles commands
-

11. ALGORITHM DEVELOPMENT

1. Start program
2. Show CLI menu
3. Take user command
4. If command = analyze:
 - Take input
 - Transform text
 - Predict result
 - Detect emotion
 - Display output
 - Save history
5. If command = history:
 - Show previous results
6. If command = exit:
 - Stop program

12. SOFTWARE ARCHITECTURE

User Input (CLI)



Command Handler

↓
Model Processing
↓
TF-IDF Vectorizer
↓
ML Model
↓
Emotion Detection
↓
Output + Storage



13. CODING

The project is implemented using Python with a modular structure:

- cli.py → CLI interaction
 - model.py → Prediction logic
 - train.py → Model training
 - utils.py → Emotion detection
-



14. OUTPUT

Example:

Enter command: analyze

Enter text: I swear I didn't do anything wrong

Truth Probability : 0.30

Deception Probability : 0.70

Emotion : Neutral

15. KEY IMPLEMENTATIONS

- TF-IDF Vectorization
 - Logistic Regression Model
 - CLI-based interaction
 - JSON-based data storage
-

16. SIGNIFICANT PROJECT OUTCOMES

- Built a complete AI-based CLI system
 - Applied ML and NLP concepts practically
 - Created a working end-to-end pipeline
 - Improved understanding of real-world AI problems
-

17. TESTING AND REFINEMENT

- Tested with multiple inputs
 - Handled invalid inputs
 - Improved CLI experience
 - Verified outputs
-

18. REAL-WORLD APPLICATIONS

- Fraud detection
 - Chat/message analysis
 - Customer feedback systems
 - Content moderation
-

19. CONTRIBUTION / FINDINGS

- Even simple models can give useful insights
 - Language patterns are important in deception
 - CLI apps are efficient and easy to use
-

20. LIMITATIONS

- Small dataset → limited accuracy
 - Basic emotion detection
 - Cannot fully understand human psychology
-

21. CONCLUSION

This project demonstrates how machine learning can be used to analyze human language and detect patterns of deception.

Although it is not perfect, it provides a strong base for future improvements and shows the practical use of AI concepts.

22. FUTURE ENHANCEMENTS

- Use advanced models like BERT 

- Improve dataset size 
 - Add voice input 
 - Add visual dashboard 
 - Deploy on cloud 
-

23. REFERENCES

- scikit-learn documentation
 - Python official docs
 - NLP tutorials and research papers
 - Online ML resources
-

FINAL NOTE

This project successfully fulfills all requirements:

- AI-based system ✓
 - CLI-based application ✓
 - Real-world problem ✓
 - Structured implementation ✓
-

 *Made with dedication by Ritesh Kumar Verma*